

# Scaling Law Report

Kanson Chu

August 8, 2026

# Contents

1. Notes
2. Exercises Writeups
  - Section 1
  - Section 2
  - Section 3
  - Section 4

Arithmetic Intensity  $\uparrow = T_{\text{mem}} > T_{\text{comp}} = \text{use most flops} = \text{compute-bound}$

Arithmetic Intensity  $\downarrow = T_{\text{mem}} < T_{\text{comp}} = \text{flops wasted} = \text{communication bound}$

peak arithmetic intensity (calculator) = peak FLOPs/s e.g. TPU v2e MXU  $\frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}} = \frac{1.97 \text{E}14}{8.2 \text{E}11} = 240 \text{ FLOPs/byte}$

\* ratio can be  $> 1$

improve performance of algorithm by increasing arithmetic intensity/memory bandwidth available

TPU [8, 128] Each position has 4 ALU

FLOPs/cycle  $8 \times 128 \times 4 = 4096$

MXU [128, 128] systolic array [256, 256] on TPU v2e  
Each position has 1 ALU

the art of scaling is figuring out how to shard our models so computation remains efficient.

4 TPUs arranged in  $2 \times 2$  grid (mesh:  $\{X: 2, Y: 2\}$ )

Col 0  $\rightarrow$  63  $\rightarrow$  64  $\rightarrow$  127 notation  
row 0 TPU 0 gets this half TPU 1  
row 1 TPU 0 TPU 1  
row 2 TPU 2 TPU 3  
row 3 TPU 2 TPU 3

\* Nothing is duplicated

global shape: (4, 128) local shape: (2, 64)

Mesh  
devices =  $((0, 1), (2, 3))$   
axis-hats =  $(\hat{x}, \hat{y})$

outer tuple: 2 items inside = 2 rows  $\times$  mesh axis

inner tuple: 2 items inside = 2 columns  $\times$  mesh axis

$\therefore 2 \times 2$  grid  $\rightarrow$  immutable list

array  $A[1, 2]$

sharding  $A[1, 2]$   $\rightarrow$  shard the first axis  $I$ , along the mesh axis  $X$ ,  
and the second axis  $J$ , along the mesh axis  $Y$ .

this tells us that each shard holds  $\frac{1}{M \times N}$  e.g.  $\frac{1}{2 \times 2} = \frac{1}{4}$

size of shard is local shape (11/2, 11/2) mesh axis

size of first dim  $\downarrow$  128

$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$   $B = \begin{bmatrix} 10 & 20 & 30 & 40 \\ 50 & 60 & 70 & 80 \end{bmatrix}$

$A[1, 2]$   $B[2, 1]$

\* Different  $X$  coordinates have different items

$X=0$   $X=1$   $X=0$   $X=1$   
 $Y=0$   $\begin{bmatrix} 1 & 3 \\ 5 & 7 \end{bmatrix}$   $Y=0$   $\begin{bmatrix} 10 & 30 \\ 50 & 70 \end{bmatrix}$   
 $Y=1$   $\begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$   $Y=1$   $\begin{bmatrix} 20 & 40 \\ 60 & 80 \end{bmatrix}$

The sharding depends on the dimension

Along  $Y$  is replicated because  $AB$  isn't sharded along  $Y$  axis

$\text{AllGather}_x [B_x, D_y] \rightarrow [B, D_y]$

data moved =  $\frac{BD}{Y}$  e.g.  $\frac{BD}{Y} \rightarrow 2 \cdot \frac{BD}{Y}$   
or the sharded dimension (if still sharded)  
Each array has a shape of

\*  $\frac{1}{Y}$  of the whole array

$T_{\text{AllGather}} = \frac{\text{bytes transferred}}{\text{Wired bandwidth}}$

Heads (1 separate copy of the Q/K/V but with its own weights)

MHA (Multi-head attention)  $N = \text{query vectors}$   $N=K$  e.g. 8 query heads  
 $K = \text{KV vectors}$  8 key and value heads

MQA (Multi-query attention)  $N = \text{a large number}$   $K=1$

GQA (Grouped-query attention)  $N=8$   $K=2$  Query 0-3 share KV 0  
Query 4-7 share KV 1

$G_r = 8/2 = 4 \text{ query heads/KV head}$

$6 \times \text{params} \times \text{hidden tokens} = \text{training flops}$

$6 \times \text{params} = \text{each token cost}$

Inference Forward and Reverse FLOPs Matrix load  
forward  $C = AB$   $2NPM$  Jay's Blog / CG Transformers  
backward (weights)  $2L$   $2NPM$   $\rightarrow$  2DF 2 matrices in FFN  
backward (activation)  $2L$   $2NPM$  training  $\rightarrow$  12BTF  
Training  $6NPM = 3 \times \text{Inference}$  Scaling Back / Modern LLM  
transposed = swapping the row and column's shape  $\rightarrow$  3DF 3 matrices in gated einsum  
e.g.  $\begin{bmatrix} 1 & 2 \end{bmatrix}$   $\begin{bmatrix} 1 & 1 \\ 2 \end{bmatrix}$

Section 4  
 $x[P]$  is  $ae$ -dimensional vector  
its shape is just  $[P] \times [P, 1]$  or  $[1, P]$

ESPP D/F (model dimensions) are sharded but  $\text{AllGather}$   
before matmul.  $B$  is sharded  
Tensor parallelism D/F are sharded

\* Element-wise operations  
2 tensors/matrices of same  
shape to multiply on matching  
positions

Dot product  
multiply + sum vectors of the  
same length  $\rightarrow$  1D arrays  
output is a scalar

Matmul (many dot products)  
 $C = AB$

latency bound  $T_{\text{min}} \times \sum_i |x_i| C_{ij} = \text{dp. of ith row}$   
was 2 axes and  $j$ th column

$\sum_i |x_i| = 4 + 4 = 8$   
each array has a shape of  
 $= 4 \times 4$

**Q1.1.**  $X[B, D], Y[D, F]$ **Need to load from memory:**  $BD + DF$ **Need to be written back to memory:**  $BF$ 2. dot product:  $D + D - 1 = 2D - 1 \approx 2D$ **Total OPs:**  $2BDF$  $[B, F]$ : output grid of numbers. Total OPs:  $2BDF$ .

$$3. \frac{2BDF}{BD + DF + BF} = \frac{\text{Total OPs}}{\text{Total Comms}}.$$

**Arithmetic intensity (for algo):**  $\frac{2BDF}{BD + DF + BF} \approx \frac{2BDF}{DF} = 2B$ .

**Arithmetic intensity (for accelerator):**  $\frac{3.94 \times 10^{14}}{8.2 \times 10^{11}} = 480$ ;  $2B > 480$ ,  $B > 240$ .

$$4. T_{\text{math}} = \frac{\text{FLOPs}}{\text{Accelerator FLOPs/s}}, \quad T_{\text{comms}} = \frac{\text{bytes}}{\text{bandwidth (bytes/s)}}.$$

**Upper bound:**  $T_{\text{math}} + T_{\text{comms}}$ .**Lower bound:**  $\max(T_{\text{math}}, T_{\text{comms}})$ .

$$T_{\text{math}} = \frac{2BDF}{3.94 \times 10^{14}}, \quad T_{\text{comms}} = \frac{BD + DF + BF}{8.2 \times 10^{11}}.$$

**Q2.**  $2B > 480$ ; **Batch size:**  $B > 240$ .

Arithmetic intensity (for algo):  $\frac{2BDF}{2BD + DF + 2BF} = \frac{2BDF}{DF} = 2B$ , hence  $2B > 240$ ,  $B > 120$ .

$$\boxed{\text{bf16}[B, D] * \text{int8}[D, F] \longrightarrow \text{bf16}[B, F]}$$

Input activations  $\rightarrow$  batch size; static  $\rightarrow$  output dimension; weights; compute.

**Q3.**  $y$ -axis: peak FLOPs/s;  $x$ -axis: batch size; numpy: average (1, 500).

$$\text{Total flops} = 2BDF, \quad T_{\text{math}} = \frac{2BDF}{1.97 \times 10^{14}},$$

$$T_{\text{comms}} = \frac{2BD + DF + 2BF}{8.2 \times 10^{11}}.$$

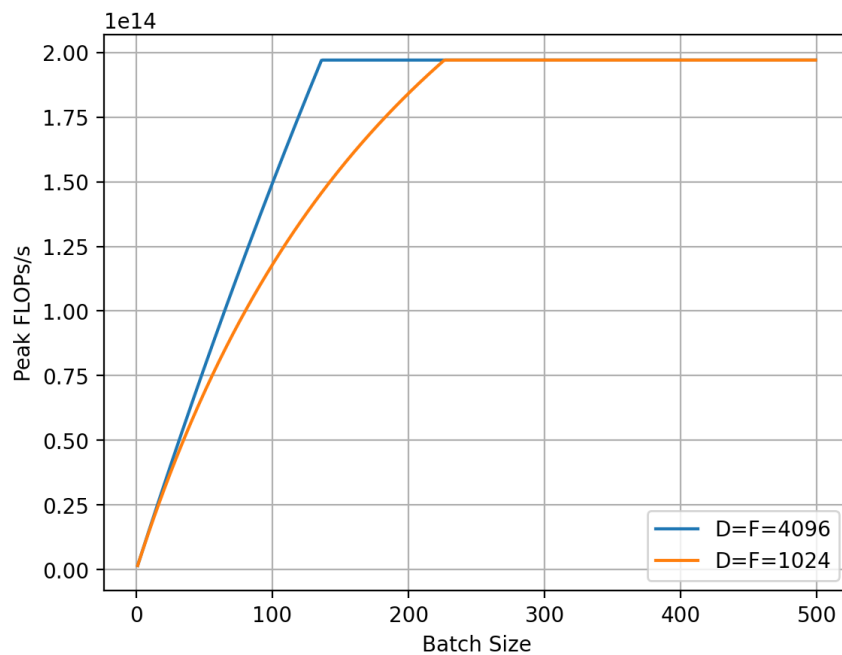
$$\text{total time} = \max(T_{\text{math}}, T_{\text{comms}}); \text{ peak FLOPs/s} = \frac{2BDF}{\text{total time}}.$$

### Python Code: Roofline Plot

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 batch_size = np.arange(1, 500)
5
6 def peak_flops_per_second(B, D, F):
7     total_flops = 2 * B * D * F
8     t_math = total_flops / 1.97e14
9     t_comms = (2 * B * D + D * F + 2 * B * F) / 8.2e11
10    total_time = np.maximum(t_math, t_comms)
11    return total_flops / total_time
12
13 roofline_4096 = peak_flops_per_second(batch_size, 4096, 4096)
14 roofline_1024 = peak_flops_per_second(batch_size, 1024, 1024)
15
16 plt.plot(batch_size, roofline_4096, label='D=F=4096')
17 plt.plot(batch_size, roofline_1024, label='D=F=1024')
18 plt.xlabel('Batch Size')
19 plt.ylabel('Peak FLOPs/s')
20 plt.legend()
21 plt.grid()

```



Peak FLOPs/s as a function of batch size for two matrix dimensions.

**Q4.** Arithmetic intensity =  $\frac{\text{total ops}}{\text{total bytes}}$ .

total ops:  $D + D - 1 \approx 2D$  (dot product);  $[1, D] \cdot [D, F] = [1, F]$ ;  $2DF \times B = 2BDF$ .

total bytes:  $BD + BDF + BF$ .

**Arithmetic intensity:**  $\frac{2BDF}{BD + BDF + BF} = \frac{2BDF}{BDF} = 2.$

**Q5.** Arithmetic intensity (H100 SXM):  $\frac{1979 \times 10^{12} \div 2}{3.35 \times 10^{12}} = 295$  FLOPs/byte.

$[B, D], D[D, F] \rightarrow [B, F]$ ; total OPs =  $2BDF$ ; total comm =  $2BD + 2DF + 2BF$ ;  $\frac{2BDF}{2DF} = B$ .

$B_{\text{crit}} \geq 295.$

**Pop Quiz Appendix A.**

$8 \times 128 \times 4 = 4096$  FLOPs/cycle;  $1.75 \times 10^9$  cycle/s;  $4096 \times 1.75 \times 10^9 = 7.168 \times 10^{12}$  FLOPs/s.

Calculate FLOPs/cycle first then times cycle/s.

**Q1.**  $200B = 200 \times 10^9$ ;  $200 \times 10^9 \times 2 = 400 \times 10^9$  B;  $400 \times 10^9 \div 32 = 1.25 \times 10^{10}$ ;  $1.25 \times 10^{10} / 1.2 \times 10^{12} = 10.4$  ms. **10.4 ms**

**Q2.** TPU v5e pod

Hosts: **32**

Tensor Cores: **256**

Total FLOPs/s (whole pod):  $5.0432 \times 10^{16}$

Total HBM: **4096 GB**

TPU v5p pod

Hosts: **2240**

Tensor Cores: **17920**

Total FLOPs/s (whole pod):  $4.11264 \times 10^{18}$

Total HBM: **860160 GB**

**Q3.**  $T_{\text{math}} > T_{\text{comms}}$ : compute bound.

$$\frac{2BDF}{9.2 \times 10^{14}} > \frac{2(BD + DF + BF)}{1.6 \times 10^{10}}.$$

With  $F = 4D$ ,

$$\frac{2BD(4D)}{9.2 \times 10^{14}} > \frac{2(BD + 4D(D) + B(4D))}{1.6 \times 10^{10}}.$$

Neglecting the activation terms  $BD$  and  $B(4D)$  relative to  $4D^2$ ,

$$\frac{8BD^2}{9.2 \times 10^{14}} > \frac{8D^2}{1.6 \times 10^{10}} \implies B > \frac{9.2 \times 10^{14}}{1.6 \times 10^{10}} = 57,500.$$

**$B > 57,500$ .**

Use prior knowledge sometimes.

$$\mathbf{Q4.(1)} \quad T_{\text{math}} = \frac{2B(4096)(16384)}{3.94 \times 10^{14}} = 3.4 \times 10^{-7} B.$$

$$T_{\text{comms}} = \frac{B(4096) + 4096(16384) + B(16384)}{8.2 \times 10^{11}}.$$

$$\max(T_{\text{math}}, T_{\text{comms}}) = \left( 3.4 \times 10^{-7} B, \frac{B(4096) + 4096(16384) + B(16384)}{8.2 \times 10^{11}} \right).$$

$$3.4 \times 10^{-7} B > \frac{4096B + 4096(16384) + B(16384)}{8.2 \times 10^{11}}; \quad B > 267.$$

$$\mathbf{(2)} \quad \text{VMEM} = \text{HBM BW} \times 22 = 8.2 \times 10^{11} \times 22 = 1.804 \times 10^{13}.$$

$B > 11.$  around 20 in practice if loaded from VMEM.

$$\text{Q5.(1)} \quad \begin{bmatrix} 0,0 & 0,1 & 0,2 & 0,3 \\ 1,0 & 1,1 & 1,2 & 1,3 \\ 2,0 & 2,1 & 2,2 & 2,3 \\ 3,0 & 3,1 & 3,2 & 3,3 \end{bmatrix} \quad \frac{2 \times 8 \times 128 \times 8192}{2 \times 4.5 \times 10^{10}} = 186 \mu s.$$

$6 \times 1 = 6 \mu s$ ;  $6 \mu s$  for first byte.

**Q5.(2)**  $186 + 6 = 192 \mu s$  total time.

**Q6.** PCIe to TPUs respectively mapping:  $(128 \times 1024)^2 \div (16 \times 1.6 \times 10^{10})$ ; each TPU has 1.07 GB.

178 ms  $\rightarrow$  TPU (0,0); all TPUs; load bf16 [8,128,1024] (2 MB);  $2.5 \times 10^{-3}$  s.

load HBM matrix to MXU: 20 ms; total 1.4 ms; **upper bound 266.4 ms; lower bound 178 ms; real case  $\approx$  200 ms.**

**Pop Quiz 1 [2D sharding across 1 axis].**

$$\left\lceil \frac{1024}{8 \times 2} \right\rceil, 4096 = [64, 4096] \text{ per device.}$$

$$64 \times 4096 \times 4 = 1,048,576 \text{ B.}$$

**Data per device (bytes): 1,048,576 B. Time to load from HBM:  $3.08 \times 10^{-7}$  s.**

**Pop Quiz.**  $\left\lceil \frac{128}{2 \times 8} \right\rceil, 2048 = [8, 2048] \text{ per device.}$

$$128 \times 2048 \times 2 = 524288 \text{ B.}$$

per device memory: 16384 B; total memory = 524288 B.

Just make sure to shard the array in every mesh axis, or else the original array will have  $k$  replicas, where  $k$  is the number of axes you did not shard. This is also known as partially-replicated.

Shard all mesh axes  $(x, y, z)$  to ensure no duplication in any place.

**Pop Quiz 2 [AllGather time].**

$$\text{Total } T_{\text{total}} = \frac{2 \times 2048 \times 8192}{9 \times 10^{10}} \approx 373 \mu s > 3 \mu s \text{ (latency bound), so not latency-bound.}$$

$$\text{Unidirectional } T_{\text{total}} = \frac{3 \times 8,388,608}{4.5 \times 10^{10}} \approx 559 \mu s.$$

$$\text{Part 2: } T_{\text{hop}} = \frac{(256/4) \cdot 256 \cdot 2}{4.5 \times 10^{10}} \approx 0.7 \mu s < 1 \mu s; \text{ total} = \max(\text{latency, bandwidth time}), 3 \cdot 1 \mu s = 3 \mu s.$$

**ideal total = 373  $\mu s$ ; unidirectional total = 559  $\mu s$ ;  $T_{\text{hop}} \approx 0.7 \mu s < 1 \mu s$ , total = 3  $\mu s$ .**

**Q1.**  $\frac{Y \cdot Z \cdot \text{size of } A}{\text{size } A} = \frac{8 \cdot 2}{1} = 16$ ; **16:1**.

**Q2.** Two equivalent layouts of a  $4 \times 4$  matrix:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Mesh axes:  $x_0, x_1, x_2, x_3$  and  $y_0, y_1, y_2, y_3$ ;  $Y_{\text{out}} = Y_{\text{inp}}$ .

If we use the idling  $z$  axis, parts 1 and 2 become shorter, e.g.  $\frac{2BD}{2YW} = 11.6 \mu s$  (almost twice faster).

$$\begin{array}{ll} \text{Pt 1:} & \frac{2BD}{YW} = \frac{2(1024)(4096)}{4(9 \times 10^{10})} \approx 23 \mu s, \quad \text{latency: } \frac{4}{2} = 2 \mu s \\ \text{Pt 2:} & \frac{2BD}{ZW} = \frac{2(1024)(4096)}{2(9 \times 10^{10})} \approx 46 \mu s, \quad \text{latency: } \frac{4+4}{2} = 4 \mu s \\ \text{Pt 3:} & 2 \cdot \frac{2BD}{XYW} = \frac{4BD}{XYW} = \frac{4(1024)(4096)}{(4)(4)(9 \times 10^{10})} \approx 11 \mu s, \quad \text{latency: } \frac{4}{2} = 2 \mu s \\ \text{Pt 2 AG}_{XYZ}: & \frac{2BD}{3W} = 31 \mu s. \end{array}$$

Real-World Practice times.

**Q3.**  $\frac{4}{2} \times 1 \mu s = 2 \mu s$ ;  $\frac{2(128)}{4} = 64$  B/device.

$T_{\text{hop}} = \frac{64}{4.5 \times 10^{10}} \approx 1.42 \times 10^{-9}$  s, negligible compared with  $1 \mu s$ .

$2 \mu s$ ;  $1 \mu s$  per hop latency;  $2 \mu s$  overall latency bound.

**Q4. Strategy 1**  $T_{\text{total}} = \max\left(\frac{2BDF}{C}, \frac{2DF}{W_{ici}}\right)$ .

$\frac{2DF}{W_{ici}} > \frac{2BDF}{C}$ ;  $\frac{C}{W_{ici}} > B$ ;  $B < 2550$  (comms bound).  
 $T_{\text{comms},2} < T_{\text{comms},1}$ :

$$\frac{4BF}{W_{ici}} < \frac{2DF}{W_{ici}} \implies 2B < D.$$

$D > 2B$ . Usually true because  $D$  is large and  $B$  is small.

$T_{\text{comms},2} < T_{\text{math},1}$ :

$$\frac{4BF}{W_{ici}} < \frac{2BDF}{C} \implies \frac{C}{W_{ici}} < \frac{D}{2} \implies D > 5100.$$

$D > 5100$ .

**Strategy 2**  $T_{\text{total}} = \max\left(\frac{2BDF}{XC}, \frac{4BF}{W_{ici}}\right)$ .

Comms bound when

$$\frac{4BF}{W_{ici}} > \frac{2BDF}{XC} \implies \frac{2}{W_{ici}} > \frac{D}{XC} \implies \frac{C}{W_{ici}} > \frac{D}{2X}.$$

$5100 > D/X$ , so  $X > D/5100$ ;  $X < 1.56 \approx 2$ .

Always comms bound almost.

### Strategy 2 is better than Strategy 1

| Batch size $B$ | Strategy 1 bound type | Strategy 2 faster if | Usually true?      |
|----------------|-----------------------|----------------------|--------------------|
| $B < 2550$     | comms bound           | $D > 2B$             | Yes                |
| $B > 2550$     | compute bound         | $D > 5100$           | Yes (large models) |

**Q5.**  $A[I, J_{XYZ}] \cdot B[J_{XYZ}, K] \rightarrow \text{AllReduce}_{XYZ}(C[I, K]\{U_{XYZ}\})$ .

Compute time =  $\frac{2IJK}{XYZC}$ ; communication time =  $\frac{4IK}{Wici}$ .

Alternatively, shard a non-contracting dimension:

$$A[I_{XYZ}, J] \cdot B[J, K] \rightarrow \text{AllGather}_{XYZ}(C[I_{XYZ}, K])$$

or

$$\boxed{A[I, J] \cdot B[J, K_{XYZ}] \rightarrow \text{AllGather}_{XYZ}(C[I, K_{XYZ}])}$$

Compute time =  $\frac{2IJK}{XYZC}$ ; communication time =  $\frac{2IK}{Wici}$ .

Sharded over all three mesh axes  $X, Y, Z$ , hence per-device FLOPs are  $1/(XYZ)$  of the unsharded FLOPs. Here  $XYZ$  in the denominator denotes the product of the mesh-axis sizes. Both AllGather alternatives produce the replicated output  $C[I, K]$ .

**Q6.** Time pairs below are  $(T_{\text{math}}, T_{\text{comms}})$ .

P1:  $A[I_X, J_Y] \cdot B[J_Y, K] \rightarrow C[I_X, K]$ ,      comm primitive:  $\text{AllReduce}_Y(C[I_X, K]\{U_Y\})$   
 $\left(\frac{2IJK}{XYC}, \frac{4IK}{XWici}\right)$

P2:  $A[I_X, J] \cdot B[J_X, K_Y] \rightarrow C[I_X, K_Y]$ ,      comm primitive:  $\text{AllGather}_X(B[J_X, K_Y])$   
 $\left(\frac{2IJK}{XYC}, \frac{2JK}{XYWici}\right)$       moving  $B$  this time, not  $C$ :  $20K \times 21K$

P3:  $A[I_X, J] \cdot B[J, K_Y] \rightarrow C[I_X, K_Y]$ ,      comms: N/A  
 $\left(\frac{2IJK}{XYC}, \text{N/A}\right)$

**Q7.** Each weight matrix ( $W_{in}$  or  $W_{out}$ ):  $\frac{2 \cdot 8192 \cdot 32768}{10^6} \approx 536$  MB; input:  $\frac{2 \cdot 128 \cdot 8192}{10^6} \approx 2$  MB.

1.  $\text{In}[B_X, D] \cdot W_{in}[D, F] \cdot W_{out}[F, D] \rightarrow \text{Out}[B_X, D]$ .

Gather the weights before their respective matmuls:

$\text{AllGather}_{XY}(W_{in}[D_{XY}, F]) \rightarrow W_{in}[D, F]$ ,  $\text{AllGather}_{XY}(W_{out}[F, D_{XY}]) \rightarrow W_{out}[F, D]$ .

Intermediate activation:  $[B_X, F]$ .

2.  $\text{In}[B, D_{XY}] \cdot W_{in}[D, F_{XY}] \cdot W_{out}[F_{XY}, D] \rightarrow \text{Out}[B, D_{XY}]$ .

$\text{AllGather}_{XY}(\text{In}[B, D_{XY}]) \rightarrow \text{In}[B, D]$ .

Intermediate activation:  $[B, F_{XY}]$ ;  $\text{ReduceScatter}_{XY}(\text{Out}[B, D]\{U_{XY}\}) \rightarrow \text{Out}[B, D_{XY}]$ .

$T_{\text{total}} = T_{\text{comms}} + T_{\text{math}}$ .

$$1: \frac{4DF}{W_{ici}} + \frac{2C}{4BDF} = \frac{4DF}{W_{ici}} + \frac{2BDF}{C}.$$

$$2: \frac{4BD}{W_{ici}} + \frac{4BDF}{4C} = \frac{4BD}{W_{ici}} + \frac{BDF}{C}.$$

**2 is better.**

**Q8. done on computer.**

### Python Code: Collective Operations

```

1 import jax
2 jax.config.update("jax_num_cpu_devices", 8)
3
4 import jax.numpy as jnp
5 import jax.sharding as shd
6 from jax import jit
7
8 # Create our mesh! We're running on a fake 8 TPU mesh (4x2) with names 'X' and 'Y'.
9 assert len(jax.devices()) == 8
10 mesh = jax.make_mesh(axis_shapes=(4, 2), axis_names=('X', 'Y'))
11
12 # A little utility function to help define our sharding. A PartitionSpec is our
13 # sharding (a mapping from axes to names).
14 def P(*args):
15     return shd.NamedSharding(mesh, shd.PartitionSpec(*args))
16
17 # We shard both A and B over the non-contracting dimension and A over the contracting dim.
18 A = jnp.zeros((8, 2048), dtype=jnp.bfloat16, device=P(None, 'Y'))
19 B = jnp.zeros((2048, 8192), dtype=jnp.bfloat16, device=P('Y', None))
20
21 # AllGather
22 @jit
23 def allgather(x):
24     return jax.device_put(x, P(None, None))
25
26 # AllReduce
27 @jit
28 def allreduce(x):
29     total = jnp.sum(x)
30     return jax.device_put(total, P())
31
32 # ReduceScatter
33 @jit
34 def reducescatter(x, y):
35     result = jnp.matmul(x, y)
36     return jax.device_put(result, P('Y', None))
37
38 # AllToAll
39 @jit
40 def alltoall(x):
41     return jax.device_put(x, P('Y', None))
42
43 print("AllGather")
44 %timeit allgather(A).block_until_ready()
45
46 print("AllReduce")
47 %timeit allreduce(A).block_until_ready()
48
49 print("ReduceScatter")
50 %timeit reducescatter(A, B).block_until_ready()
51
52 print("AllToAll")
53 %timeit alltoall(A).block_until_ready()

```

### Terminal Output: Collective Timings

```

1 AllGather time:
2 7.57 ms ±287 µs per loop (mean ±std. dev. of 7 runs, 100 loops each)
3
4 AllReduce time:
5 15.8 ms ±2.53 ms per loop (mean ±std. dev. of 7 runs, 100 loops each)
6
7 ReduceScatter w/ MatMul time:
8 383 ms ±16.4 ms per loop (mean ±std. dev. of 7 runs, 1 loop each)
9
10 AllToAll time:
11 8.04 ms ±1.04 ms per loop (mean ±std. dev. of 7 runs, 100 loops each)

```

**Q9.(1)**  $o_{kj} = \sum_i a_{ki} b_{ij}$ .

**(2)**  $\text{ReduceScatter}_{X,K}(C[N, K]\{U_X\}) \rightarrow C[N, K_X]$ .

**(3)**  $\frac{NK}{Wici} : \frac{NM}{Wici}; K : M$ .

**Q10.(1)** size sent in one step:  $\frac{N}{D} \times N = \frac{N^2}{D}$ ; number of steps:  $D - 1$ .

total scalars:  $\frac{(D-1)N^2}{D}$ .

**(2)** size sent in one step:  $\frac{N}{D} \times \frac{N}{D} = \frac{N^2}{D^2}$ ; number of steps:  $\frac{(D-1)D}{2}$ .

total scalars:  $\frac{N^2}{D^2} \cdot \frac{(D-1)D}{2} = \frac{N^2}{2} \cdot \frac{D-1}{D}; \frac{N^2}{2} \left(1 - \frac{1}{D}\right)$ .

**(3)**  $(1):(2) = N^2 \left(1 - \frac{1}{D}\right) : \frac{N^2}{2} \left(1 - \frac{1}{D}\right); 2 : 1$ .

**(4)**  $\frac{N^2}{D} \cdot \frac{D}{2} = \frac{N^2}{2} \cdot \frac{N^2}{2}$ .

**(5)** With  $m = D/2$ , the number of steps is

$$\frac{m(m+1)}{2} = \frac{(D/2)(D/2+1)}{2} \approx \frac{D^2}{8}.$$

Total scalars:

$$\frac{N^2}{D^2} \cdot \frac{(D/2)(D/2+1)}{2} = \frac{N^2}{8} \left(1 + \frac{2}{D}\right) \approx \frac{N^2}{8}.$$

**Approximately  $\frac{N^2}{8}$  for large  $D$ .**

**(6)** Using the approximation in (5),  $(4):(5) \approx \frac{N^2}{2} : \frac{N^2}{8}$ ; **approximately 4 : 1.**

**Factor of 2 comes from the closed-form formula  $n(n-1)/2$ .**

**Q1.**  $L(3DF + 4DNH + 2D) + 2DN$ . With  $L = 64$ ,  $D = 4096$ ,  $F = 4(4096)$ ,  $NH = 4096$ , and  $N = 32000$ ,

$$64(3 \cdot 4096 \cdot 4 \cdot 4096 + 4 \cdot 4096 \cdot 4096 + 2 \cdot 4096) + 2 \cdot 4096 \cdot 32000 \approx 1.7 \times 10^{10}.$$

**Approximately 17B parameters.**

Attention parameters as a fraction of the MLP plus attention parameters:

$$\frac{4DNH}{3DF + 4DNH} = \frac{4 \cdot 4096 \cdot 4096}{3 \cdot 4096 \cdot 4 \cdot 4096 + 4 \cdot 4096 \cdot 4096} = 0.25 = \frac{1}{4}.$$

KV cache per token:  $2LKH = 2 \cdot 64 \cdot 4096 = \mathbf{524,288 \text{ B}}$ .

**Q2.** Total FLOPs =  $2BDF$ . The total actual FLOPs are  $2BDFZ$ . The FLOPs per TPU may be obtained either by dividing the total FLOPs by the sharded axes,

$$\frac{2BDF}{XY},$$

or by dividing the total actual FLOPs by all mesh axes:

$$\frac{2BDFZ}{XYZ} = \frac{2BDF}{XY}.$$

**Q3.**  $A[I, J, K, L] \cdot B[I, J, M, N, O] \longrightarrow C[K, L, M, N, O]$ . The contracting dimensions are  $I$  and  $J$ , so **the total FLOPs are  $2IJKLMNO$ .**

**Q4.** Arithmetic intensity is  $\frac{\text{total FLOPs}}{\text{total bytes}}$ . For attention,

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^T)V.$$

The two matmuls each cost  $2HBT SN$  FLOPs, while softmax costs  $O(BTSN)$ . Thus

$$\text{total FLOPs} = 2HBT SN + 2HBT SN + O(BTSN) \approx 4HBT SN.$$

The bytes moved are

$$2BTNH + 2BSKH + 2BSKH + 2BTNH = 4BTNH + 4BSKH = 4BHK(TG + S),$$

where  $N = KG$ . Therefore

$$\text{AI} = \frac{4HBT SN}{4BHK(TG + S)} = \frac{TG}{TG + S}.$$

For  $T = S$ ,  $\text{AI} = \frac{T^2G}{TG + T} = \frac{TG}{G + 1} = O(T)$ . For  $T \ll S$ ,  $\text{AI} = \frac{SG}{G + S} \approx \mathbf{G}$ .

**Q5.** Equating attention FLOPs with QKVO projection FLOPs gives

$$12BT^2NH = 24BTDNH \implies \mathbf{T=2D}.$$

**Q6.** For the  $QK^T$  calculation,

$$\frac{6BTSNH}{3} = 2BTSNH = 2BT^2NH.$$

The  $\text{softmax}(QK^T)V$  calculation also costs  $2BT^2NH$  FLOPs.

**Total:  $4BT^2NH$  FLOPs.**

**Q7.** Peak FP8 performance with structured sparsity is 3026 TFLOPs/s, or 1513 TFLOPs/s without it:

$$1.513 \times 10^{15} \text{ FLOPs/s.}$$

Over  $2.79 \times 10^6$  accelerator-hours,

$$1.513 \times 10^{15} \cdot 2.79 \times 10^6 \cdot 60 \cdot 60 = 1.52 \times 10^{25} \text{ total FLOPs.}$$

$$\frac{6 \cdot 37 \times 10^9 \cdot 19.5 \times 10^{12}}{1.52 \times 10^{25}} = \mathbf{21.6\%}.$$

**Q8.** TPU arithmetic intensity:

$$\frac{1.97 \times 10^{14}}{8.2 \times 10^{11}} \approx 240.$$

For  $\frac{2KBDF}{FDF} > 240$ ,

$$\frac{2KB}{F} > 240 \implies B > \frac{120F}{K}.$$

For DeepSeek,  $F = 256$  and  $K = 8$ , so  $B > \frac{120(256)}{8} = 3840$ .